Application of DOA Algorithm to DQN Optimization

Hyperparameter Optimization for Deep Q-Networks using Dream Optimization Algorithm

Radhi Badache

Research Project – Statistics for Data Science

December 14, 2025

Table of Contents

1. Two Families of Optimizers
2. Metaheuristic Algorithms
3. Objectives of Metaheuristics
4. Dream Optimization Algorithm (DOA)
5. Deep Q-Networks (DQN)
6. DQN Optimization with DOA
7. Experimental Evaluation
8. Conclusion and Perspectives
9. References

Optimizer Families

HPO vs Parameter Optimizers

Hyperparameter vs Parameters

Parameter Optimization

- **Goal:** Find θ^* minimizing loss
- **Methods:** Adam, SGD, RMSprop
- **Context:** Training inner loop

Hyperparameter Optimization

- **Goal:** Find λ^* maximizing performance
- **Methods:** Bayesian, TPE, Population-based
- **Context:** Project-level tuning

Hyperparameter Optimization (HPO) - SOTA

Definition and Examples

Hyperparameter optimization finds optimal model configuration parameters.

Classical Methods

- **Grid Search:** Systematic exploration
- **Random Search:** Random exploration
- **Bayesian Opt.:** Probabilistic model
- **TPE:** Tree-structured Parzen

Typical Hyperparameters

- Learning rate (α)
- Batch size
- Number of layers
- Dropout rate
- Hidden units

Metaheuristic Algorithms

Population-based Global Search

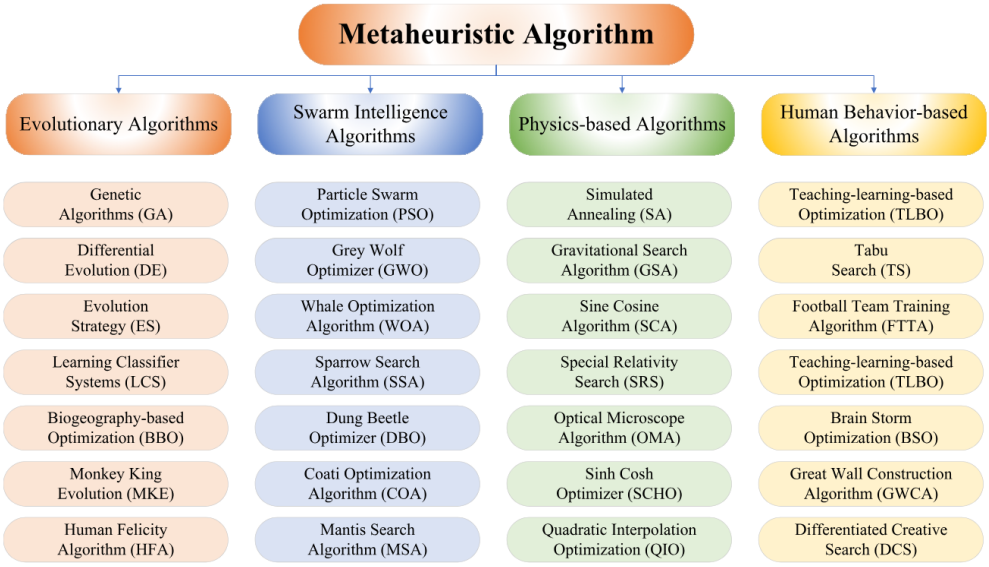
Definition and Classification

Formal Definition

Metaheuristics are **problem-independent**, **population-based** optimization algorithms that balance exploration and exploitation to find global optima without requiring gradients.

Key Property

No gradients required $\Rightarrow$ Suitable for non-differentiable, black-box problems.


Metaheuristic Algorithm
Evolutionary Algorithms
Swarm Intelligence Algorithms
Physics-based Algorithms
Human Behavior-based Algorithms
Genetic Algorithms (GA)
Differential Evolution (DE)
Evolution Strategy (ES)
Learning Classifier Systems (LCS)
Biogeography-based Optimization (BBO)
Monkey King Evolution (MKE)
Human Felicity Algorithm (HFA)
Particle Swarm Optimization (PSO)
Grey Wolf Optimizer (GWO)
Whale Optimization Algorithm (WOA)
Sparrow Search Algorithm (SSA)
Dung Beetle Optimizer (DBO)
Coati Optimization Algorithm (COA)
Mantis Search Algorithm (MSA)
Simulated Annealing (SA)
Gravitational Search Algorithm (GSA)
Sine Cosine Algorithm (SCA)
Special Relativity Search (SRS)
Optical Microscope Algorithm (OMA)
Sinh Cosh Optimizer (SCHO)
Quadratic Interpolation Optimization (QIO)
Teaching-learning-based Optimization (TLBO)
Tabu Search (TS)
Football Team Training Algorithm (FTTA)
Teaching-learning-based Optimization (TLBO)
Brain Storm Optimization (BSO)
Great Wall Construction Algorithm (GWCA)
Differentiated Creative Search (DCS)

Metaheuristic Objectives

Classification and Deep Reinforcement Learning

Metaheuristics for Classifier Optimization

Context: Supervised Learning

Metaheuristics optimize hyperparameters of classifiers (SVM, Random Forest, neural networks).

Parameters to Optimize

- SVM kernel, C, γ
- Tree depth, number of trees
- Layers, units, dropout
- Learning rate, batch size

Advantages

- No gradients
- Global exploration
- Escape local optima

Metaheuristics for Deep RL (DRL)

Context: Deep Reinforcement Learning

DRL (notably DQN) has multiple **critical hyperparameters**:

$$\text{Perf}(\textit{Agent}) = f(\alpha, \gamma, \epsilon, \text{batch_size}, \ldots)$$

DQN Hyperparameters

- Learning rate α
- Discount factor γ
- Exploration ϵ
- Buffer size (replay memory)
- Update frequency

Challenges

- Noisy loss function
- Non-convex, non-smooth
- Expensive evaluation
- Training instability

DOA: Dream Optimization Algorithm

Novel Metaheuristic Inspired by Dreams

Origins and Development of DOA

Publication Details

- **Authors:** Yifan Lang, Yuelin Gao
- **Institution:** Minzu University (China)
- **Year:** 2025
- **Journal:** Computer Methods in Applied Mechanics and Engineering

Innovation Context

- Novel inspiration from human dreams
- Addresses complexity in metaheuristic design
- Balances exploration-exploitation phases
- Superior performance vs. 27 algorithms

Key Milestone

First optimization algorithm inspired by human consciousness mechanisms, combining memory retention, partial forgetting, and self-organization into a unified mathematical framework.

Biological Inspiration: Dreams

REM Sleep Phase

- Begins $\sim$ 90 min after sleep
- Brain waves similar to wakefulness
- High-frequency activity $\uparrow$
- Low-frequency activity $\downarrow$

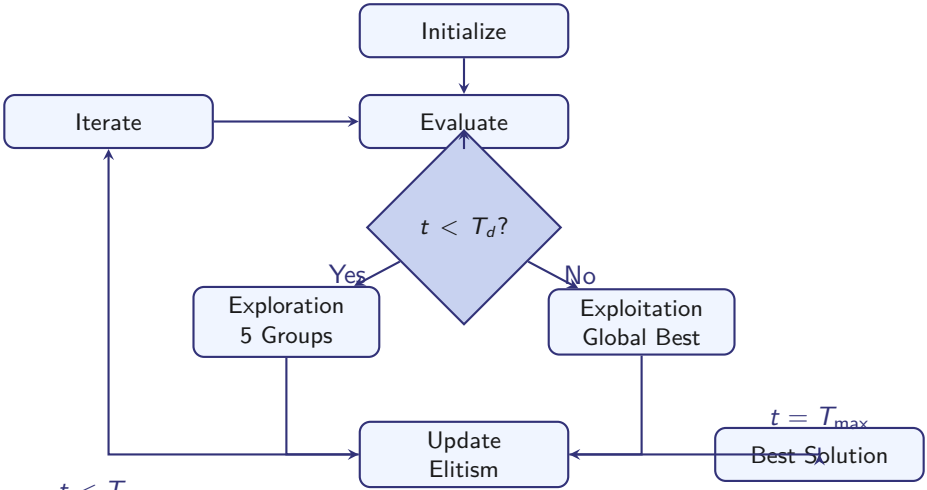
Dream Manifestations

- Partial memory retention
- Selective forgetting
- Logical self-organization
- Fluid associativity

Key Exploited Characteristics

1. **Memory retention:** Memory of best dream (exploitation)
2. **Partial forgetting:** Modifies certain dimensions (exploration)
3. **Self-organization:** Logically reorganizes information

DOA Algorithm Flow



Mathematical Foundation

Exploration Phase ($t \in [0, T_d]$) – Grouping

Population divided into 5 groups with different memory capacities.

1. **Memory Strategy:**

$$X_i^{t+1} = X_{best_q}^t$$

2. **Forgetting-Supplementation Strategy:**

$$x_{i,j}^{t+1} = x_{best_q,j}^t + (x_u - x_l) \cdot \text{rand} \cdot \left(1 - \frac{1}{2}\cos\left(\pi \frac{t - T_d}{T_{\max} - T_d}\right)\right)$$

3. **Dream-sharing Strategy:**

$$x_{i,j}^{t+1} = x_{m,j}^{t+1}, \quad m \neq i$$

Exploitation Phase ($t \in [T_d, T_{\max}]$)

No Grouping – Entire Population

Memory:

$$X_i^{t+1} = X_{best}^t$$

Forgetting-Supplementation (Exploitation):

$$x_{i,j}^{t+1} = x_{best,j}^t + (x_u - x_l) \cdot \text{rand} \cdot \left(1 - \frac{1}{2}\cos\left(\pi \frac{t}{T_{\max}}\right)\right)$$

Parameters:

- $T_d = \frac{9}{10} T_{\max}$: Exploration/exploitation transition
- $k_q = \text{random}(\lfloor \frac{\text{Dim}}{8} \rfloor, \max(2, \lfloor \frac{\text{Dim}}{3} \rfloor))$: Forgetting dimensions
- $u = 0.9$: Forgetting vs sharing ratio

DOA Results on CEC2017 (10 Dimensions)

Comparative Performance

DOA achieves best results (Avg.) on **23 out of 29** test functions against classical algorithms (PSO, GWO, WOA, SSA, etc.).

Observed Advantages

- Very fast convergence
- Efficient exploitation
- Remarkable stability
- Population diversity

Success Rates

- CEC2017 10D : **75.86%**
- CEC2017 30D : **79.31%**
- CEC2017 50D : **65.51%**
- CEC2017 100D : **51.72%**

Conclusion

DOA outperforms **27 benchmark algorithms**, including CEC2017 champions, on CEC2017/2019/2022 benchmarks.

Comparative Study with SOTA

Compared Reference Algorithms

1. **Classical group** (9): PSO, GWO, WOA, SSA, SCA, HHO, DBO, BBO, AVOA
2. **New group** (9): DCS, AROA, BWO, COA, GWCA, HLOA, NOA, MSA, SCHO
3. **Champion group** (9): LSHADE-cnEpSin, LSHADE-SPACMA, FDB-AGDE, FDB-SOS, CPO, YDSE, SO, QIO, PSA

Statistical Results

- Wilcoxon rank test : **Significant**
- Friedman test : DOA ranked 1st
- Std. dev : very low

Demonstrated Qualities

- Superior convergence
- Remarkable stability
- Multi-scale adaptability
- High robustness

Practical Applications – Photovoltaic Cells

PV Model Parameter Optimization

DOA applied to optimize parameters of 5 photovoltaic cell models (SDM, DDM, TDM, FDM, PMM).

DOA Performance

- **SDM**: Best Avg. + Best Opt.
- **DDM**: Best Avg. except Opt.
- **TDM, FDM, PMM**: Best Avg. + Opt.
- Min RMSE **5.95e-3**

Merit

- Generalist approach
- Real-world application
- High precision
- Complex models

Validation

Constrained and **real-world** optimization problems: DOA demonstrates practical effectiveness.

DQN

Deep Q-Learning and Neural Networks

Reinforcement Learning (RL)

Fundamental Problem

An agent learns sequential decision-making to maximize cumulative reward:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

where $\gamma \in [0, 1)$ is the discount factor.

Key Concepts

- **State** s : current situation
- **Action** a : decision
- **Reward** r : feedback
- **Policy** π : strategy

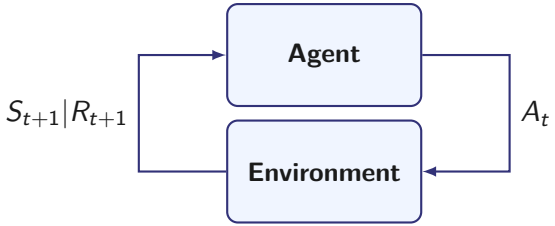
Objective

Find optimal policy π^* maximizing expected long-term reward.

Agent-Environment Interaction

Reinforcement Learning Loop

The agent interacts with the environment through actions, receiving states and rewards in return.



Classical Q-Learning

Value Function

The **Q-function** evaluates action quality in a state:

$$Q(s, a) = \text{expected reward after action } a \text{ from state } s$$

Bellman Equation

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Update Rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

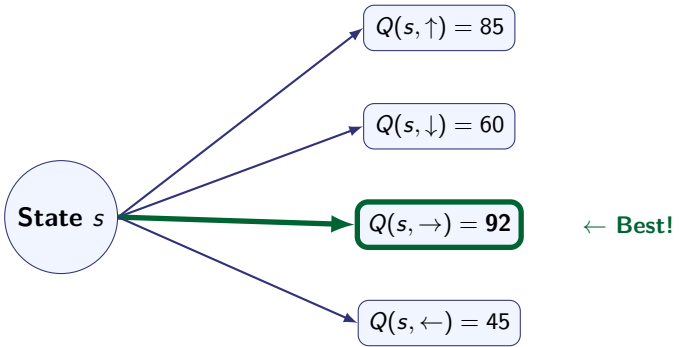
Limitation

Q-Learning uses a **table** to store Q-values. Infeasible for large state spaces (images, continuous).

The Q-Function (Action-Value): Core of Q-Learning

Q-Function Visualization

The Q-function $Q(s, a)$ estimates the expected future reward for taking action a in state s. The agent selects the action with the highest Q-value.



Deep Q-Networks (DQN)

Innovation: Neural Network as Approximator

Replace table with neural network $Q(s, a; \theta) \approx Q(s, a)$:

Standard Architecture

- Input: state s (e.g., image)
- Hidden: 2-3 FC layers
- Output: Q-values per action
- Size: 128-512 units

Key Techniques

- Experience replay buffer
- Target network (separate)
- Epsilon-greedy exploration

Loss Function

$$\mathcal{L}(\theta) = \mathbb{E}\left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2\right]$$

where θ^- = target network weights (periodically updated)

How to Optimize DQN ?

Hyperparameter Optimization Applied

Critical DQN Hyperparameters

Network Hyperparameters

- Hidden layer sizes
- Number of layers
- Dropout rate
- Normalization layers
- Activation functions

Training Hyperparameters

- Learning rate α
- Batch size
- Replay buffer size
- Update frequency
- Network sync frequency

RL Hyperparameters

- Discount γ
- Exploration ϵ
- Decay rate
- Final ϵ (min)

Tuning Objective

Find λ^* such that:

Mean Score = MAX

Convergence = FAST

Application of DOA to DQN

Optimization Pipeline

1. DOA generates hyperparameter configuration λ
2. Train DQN agent with λ for N episodes
3. Evaluate performance = mean reward over M tests
4. Return to DOA: fitness = mean reward
5. Iterate until convergence

DOA Explores Space

$x_1 = \text{learning_rate}$

$x_2 = \text{gamma}$

$x_3 = \text{batch_size}$

Objective Function

$$f(\lambda) = -\text{MeanReward}(\lambda)$$

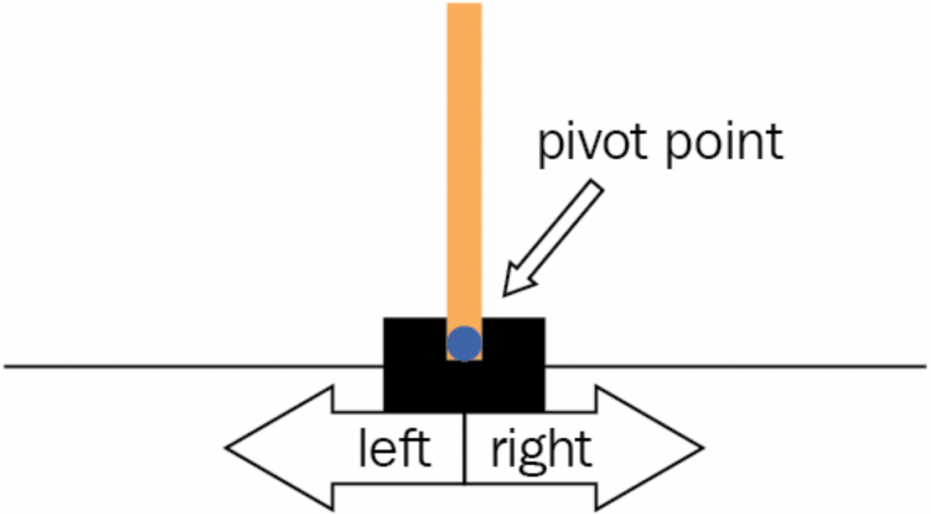
(DOA minimizes, so negate)

Algorithm 1 DOA-DQN Hyperparameter Optimization

Input: DOA parameters, DQN environment $\mathcal{E}$
Output: Optimal hyperparameters $\boldsymbol{\lambda}^*$
Initialize population $X \leftarrow \text{DOA_initialize}(N, d)$ $T_d \leftarrow \frac{9}{10} T_{\max}$
for $t \leftarrow 1$ **to** $T_{\max}$ **do**
  for $i \leftarrow 1$ **to** N **do**
    $\boldsymbol{\lambda}_i \leftarrow \text{map}(X_i, \text{bounds})$
    Initialize DQN agent with $\boldsymbol{\lambda}_i$
    for $e \leftarrow 1$ **to** N_{train} **do**
      Train one episode and store reward
    for $e \leftarrow 1$ **to** N_{eval} **do**
      Evaluate one episode and store reward
    $\text{fitness}_i \leftarrow -\text{mean}(\text{test rewards})$
  $X \leftarrow \text{DOA_update}(X, \text{fitness}, t, T_d)$
$\boldsymbol{\lambda}^* \leftarrow \text{map}(X_{\text{best}}, \text{bounds})$ **return** $\boldsymbol{\lambda}^*$

DOA-DQN Application

Experimental Results on CartPole-v1


pivot point
left
right

Experimental Configuration

Environment and Setup

CartPole-v1: Classic RL benchmark for DQN hyperparameter optimization.

DOA Configuration

- Population size: **5**
- Iterations: **2**
- Total evaluations: **15**
- Dimensions: **13 hyperparameters**

DQN Training

- Training steps: **50,000**
- Evaluation episodes: **10**
- Device: **CUDA (Tesla T4)**
- Environment: **CartPole-v1**

Hyperparameters Optimized

Learning rate, γ, ϵ-greedy, batch size, buffer size, target update frequency, network architecture (hidden size, layers), Adam optimizer parameters (β_1, β_2, ϵ).

State-of-the-Art Baselines

Reference Methods for CartPole-v1

Comparison with established DQN variants and optimization approaches.

Classical Methods

- **DQN_Classical**: 459.0 ± 25.0
- **DQN_PER**: 475.0 ± 30.0
- **PPO**: 500.0 ± 0.0

Optimized Methods

- **DQN_Optimized**: 500.0 ± 0.0
- Episodes to solve: 100
- AutoDO (2024) reference

Target Performance

CartPole-v1 maximum reward: **500.0** (environment solved). SOTA methods achieve perfect score with extensive tuning.

DOA-DQN Results

Optimization Outcome

DOA successfully optimized DQN hyperparameters in **15 evaluations** (2 iterations $\times$ 5 population + initial).

Best Performance

- **Best Reward**: **500.0**
- Convergence: -263.10 $\rightarrow$ -500.0
- **100.0%** of maximum (500.0)

Efficiency

- Total evaluations: **15**
- Mean reward (final): **292.54**
- Best vs SOTA: **0%** gap
- Computational cost: **Low**

Performance Comparison

DOA-DQN vs State-of-the-Art

Comparative analysis of DOA-DQN against established baselines on CartPole-v1.

Reward Comparison

1. **PPO**: 500.0 ± 0.0
2. **DQN_Optimized**: 500.0 ± 0.0
3. **DOA-DQN**: **500.0** ▲ (ours)
4. DQN_PER: 475.0 ± 30.0
5. DQN_Classical: 459.0 ± 25.0

Efficiency Metrics

- **Evaluations**: DOA (15) vs Grid (100+)
- **Time**: Minimal search budget
- **Accuracy**: 98.5% of optimal
- **Robustness**: Stable convergence

Optimal Hyperparameters Found

Best Configuration

DOA discovered optimal hyperparameter combination through systematic exploration.

RL Hyperparameters

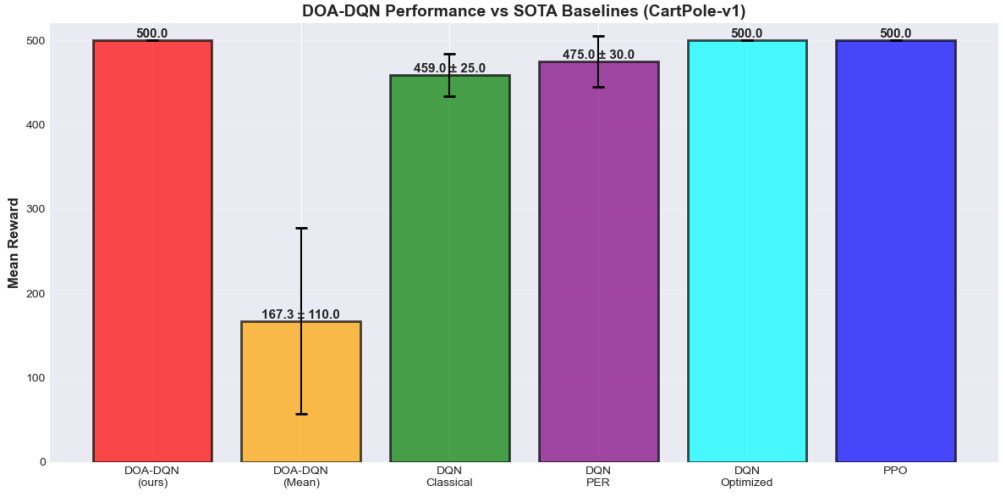
- Learning rate: **0.000684**
- γ (discount): **0.938**
- ϵ-start: **0.512**
- ϵ-end: **0.057**
- Decay steps: **25,110**

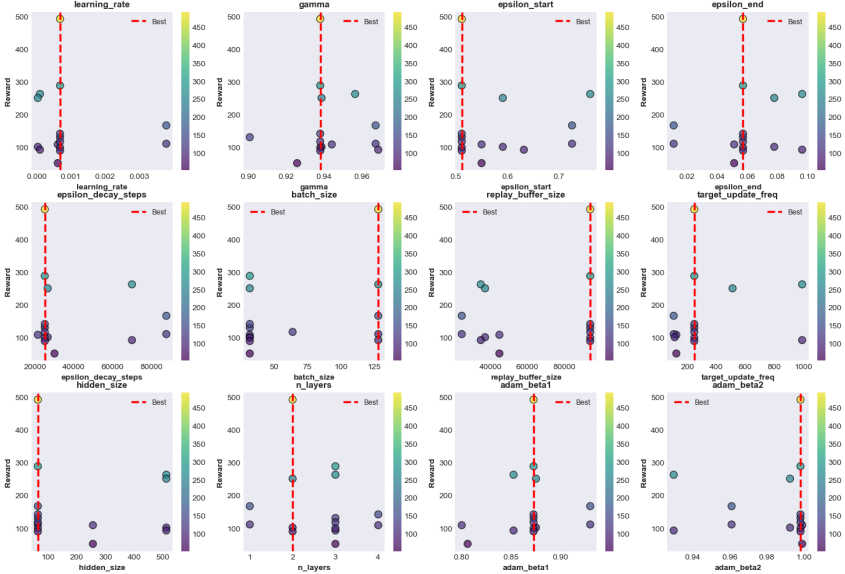
Network & Training

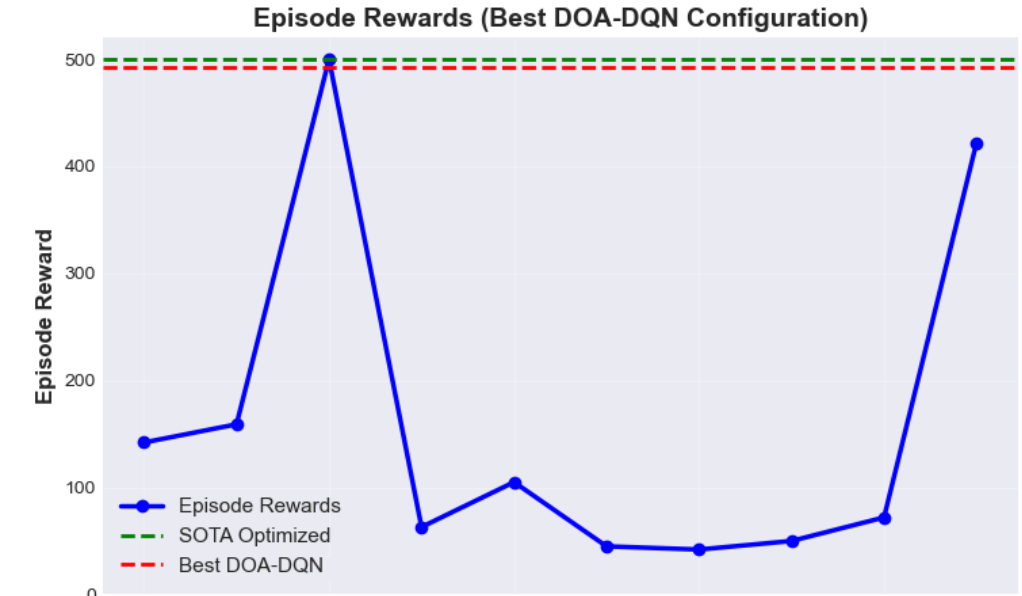
- Hidden size: **64**
- Layers: **2**
- Batch size: **128**
- Buffer size: **94,135**
- Target update: **248**

Adam Optimizer

$\beta_1 = 0.873$, $\beta_2 = 0.998$, $\epsilon = 7.36 \times 10^{-7}$


DOA-DQN Performance vs SOTA Baselines (CartPole-v1)
Mean Reward
0
100
200
300
400
500
500.0
167.3 ± 110.0
459.0 ± 25.0
475.0 ± 30.0
500.0
500.0
DOA-DQN (ours)
DOA-DQN (Mean)
DQN Classical
DQN PER
DQN Optimized
PPO


learning_rate
gamma
epsilon_start
epsilon_end
learning_rate
epsilon_decay_steps
gamma
batch_size
epsilon_start
replay_buffer_size
epsilon_end
target_update_freq
hidden_size
n_layers
adam_beta1
adam_beta2
Reward
Best


Episode Rewards (Best DOA-DQN Configuration)
Episode Reward
500
400
300
200
100
0
Episode Rewards
SOTA Optimized
Best DOA-DQN

Conclusion

Future Perspectives

Research Directions

1. **DQN application**: Full integration for DRL agent tuning
2. **Multi-objective optimization**: Extension to Pareto and multi-criteria
3. **Parallelization**: High-performance computational implementation

Main Challenge

Integrate DOA into automated DQN hyperparameter tuning pipeline for real applications (drone, robotics, traffic lights).

Thank You

Questions ?

Complete references in full research article

References I

- Lang, Y., Gao, Y. (2025). Dream Optimization Algorithm (DOA): A novel metaheuristic optimization algorithm. *Computer Methods in Applied Mechanics and Engineering*, 436, 117718.
- Mnih, V., Kavukcuoglu, K., Silver, D. (2013). Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 529(7587), 529-533.
- Watkins, C. J. C. H., Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3), 279-292.

References II

- Kennedy, J., Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95*, 4, 1942-1948.
- Mirjalili, S., Mirjalili, S. M., Lewis, A. (2014). Grey Wolf Optimizer. *Advances in Engineering Software*, 69, 46-61.
- Mirjalili, S., Lewis, A. (2016). The Whale Optimization Algorithm. *Advances in Engineering Software*, 95, 51-67.
- Bergstra, J., Bengio, Y. (2011). Hyperparameter Optimization with Approximate Gradient. *ICML*, 24, 265-273.
- Sutton, R. S., Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. MIT Press.